Day 15 Student Notes: Checkpoint, Containers and Infrastructure as Code

> **Today in one line:** use what we learned in Days 11 to 14 on one small project, and **name the evidence for everything Claude writes**.

Revision: Days 11 to 14

Day	Remember
11 Docker	A Dockerfile builds an image; a container is a running copy. Lean: slim base, multi-stage, **non-root**, .dockerignore, no secrets. Will not start: **docker ps -a**, **docker logs**.
12 Kubernetes	Deployment keeps pods running, Service gives one address. Status word, then **describe** (Events) and **logs**. Review: tag, limits, probes, non-root, secrets, replicas.
13 Ansible	Playbooks configure servers. Use **modules**, not shell. Run twice: the second run shows **changed=0**.
14 Terraform	Creates cloud infrastructure from code. **Plan first.** Read every **-** and **-/+**. Drift. Modules, and **moved** blocks when refactoring.

The loop for every tool: Claude drafts, you check the evidence.

The project: ship the shop-status app

```
app.py (the Day 11 app)
   |  Docker       -> image shop-status:1.0
   |  Kubernetes   -> 2 pods + 1 Service
   |  Terraform    -> an image registry (ECR)
```

Set up: unzip shop-checkpoint.zip, open the folder in VS Code, open Claude Code. Docker Desktop is optional. No cluster and no AWS account needed. Nothing is applied.

Your evidence for each tool

Tool	Evidence
Docker	Image size; **docker exec shop id** (not root); no .env inside
Kubernetes	The six-point checklist: pass or fail
Terraform	The plan: every destroy, every open permission

Steps and prompts (also in Day15_Prompts_To_Paste.txt)

Step 1: Claude writes the Dockerfile

```
Write a Dockerfile for this app.
Keep the image small. Run as a non-root user with ID 10001. Do not copy secrets.
Also write a .dockerignore.
```

Check: slim base with a version, packages before code, **USER 10001** (a number, so Kubernetes can check it), .dockerignore keeps .env out, no ENV secrets. Optional, with Docker:

```
docker build -t shop-status:1.0 .
docker run -d --name shop -p 5000:5000 shop-status:1.0
curl localhost:5000
docker exec shop id
docker exec shop ls -a /app
docker rm -f shop
```

You should see a small image, JSON from curl, **uid=10001** and no .env.

Step 2: Claude writes the Kubernetes files

```
Write a Kubernetes Deployment and Service for this app in a folder called k8s.
Image shop-status:1.0, port 5000, 2 replicas. The Service listens on port 80.
Add requests and limits, a readiness probe on /health, and run as user 10001 (non-
root).
```

Check: Deployment and Service, replicas 2, Service port 80 to targetPort 5000, matching labels, a probe on /health. Then make Claude review its own work:

```
Review the files in the k8s folder.
Check: pinned image tag, requests and limits, probes, non-root, no secrets, more
than 1 replica.
Say pass or fail for each, with one line on why.
```

Claude can say pass when it is not. Open the YAML yourself.

Step 3: Claude writes a Terraform module

```
Write a small Terraform module in a folder called terraform for an AWS ECR
repository
that will hold this app's image.
Inputs: name, scan on push, tags. Output: the repository URL. Image tags must not
be changeable.
```

Check: variables for name, scan on push and tags; an aws_ecr_repository resource; tags that cannot be overwritten (IMMUTABLE); an output for the repository URL; no keys in the code.

Step 4: review a teammate's Terraform

```
Review teammate/ecr.tf and teammate/plan.txt.
List every risk, most serious first, with one line on why. Is it safe to apply?
Do not change any file.
```

Find the plan summary line, Principal and force_delete first. You should find **four** risks: Principal "*" (anyone on AWS can push images), tags that can be overwritten (MUTABLE), force_delete (the repo is deleted even with images in it), and scan on push turned off. Safe to apply? **No.**

Where AI-written code needs the most scrutiny

Tool	Watch closely
Docker	Secrets in layers, root user, unpinned base image
Kubernetes	Privileged, no limits or probes, secrets in YAML
Ansible	Shell tasks that are not idempotent, passwords, chmod 777
Terraform	Destroy and replace, open access (0.0.0.0/0, Principal *), wide IAM
All	Invented versions and IDs, and how big the damage is if it is wrong

My prompt library

Open **prompt-library-template.md** and let Claude help you fill it in:

```
Help me fill in prompt-library-template.md using the prompts we used today.
For each tool keep my prompt short and add one thing I always check.
```

For each tool keep one short prompt that worked and one thing you always check. Keep this file and reuse it.

Check yourself

1. Why does the Dockerfile use USER 10001 and not a name?
2. Name three checks for a Kubernetes Deployment.
3. How do you prove an Ansible playbook is idempotent?
4. What do you read first in a Terraform plan?
5. Claude says everything passes. What do you do?

Answers

1. Kubernetes can only verify a numeric user when runAsNonRoot is set.
2. Any three of: pinned tag, requests and limits, probes, non-root, no secrets in YAML, more than one replica.
3. Run it twice. The second PLAY RECAP shows changed=0.
4. The summary line, then every - and -/+.
5. Open the files and find the evidence yourself.

Take-home task (30 minutes)

1. Finish your prompt library. For each section add the last check you learned this week.
2. Optional: write one more prompt for a tool you use at work, using fake data only.

Everything here is made up. Never paste keys, secrets or real state files into Claude.

> **Next class, Day 16:** Claude for Monitoring: Prometheus and Grafana.